AgenticData: A Universal Data Comprehension Engine with Schema Inference, Lineage Tracking, and Quality Assurance

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 2026

Abstract

Data processing tools solve individual format problems—CSV parsers read columns, SQL tools query tables, PDF extractors pull text—but none provides unified *comprehension*: understanding what data means across formats, tracking where it came from, and assessing whether it can be trusted. We present AgenticData, a universal data comprehension engine that addresses three fundamental problems in data management: *schema opacity* (not knowing what fields represent), *lineage blindness* (not tracking data provenance), and *quality silence* (not detecting degradation). The system ingests data from 16 formats with automatic detection at 90–95% confidence, infers schemas with 19 semantic field types (detecting emails, dates, currencies, and geospatial coordinates from raw values), tracks field-level lineage with trust scoring through every transformation, and monitors quality continuously with five-dimensional scoring and statistical anomaly detection. We implement AgenticData in 9,196 lines of Rust across four crates, exposing 122 MCP tools through the Model Context Protocol [3]. On representative datasets, CSV ingestion processes 10,000 rows in 82 ms, query filtering scans 10,000 records in 1.8 ms, and spatial radius queries over 1,000 points complete in 25 μs. The `.adat` binary format achieves 136.5 bytes/record with LZ4 [6] compression and BLAKE3 [15] integrity verification. All 265 tests—including 9 dedicated paper validation tests that verify every quantitative claim—pass with zero failures.

1 Introduction

The transformer architecture [19] has enabled language model agents capable of multi-step planning, autonomous task execution, and structured data analysis. Yet these agents lack a fundamental capability that even simple scripts possess: the ability to *comprehend* arbitrary data. When an agent encounters a CSV file, it can parse columns but cannot determine that column 3 contains email addresses. When it transforms data through a pipeline, it cannot trace which input produced which output. When data quality degrades silently over weeks, no alarm fires until a downstream report breaks.

The landscape of data processing has evolved considerably over the past two decades, but each tool has carved out a narrow niche. pandas [13] pioneered programmatic tabular analysis in Python but infers only primitive types and provides no lineage tracking. Apache Spark [5] parallelized transformation pipelines but requires a JVM cluster and provides only table-level provenance. Great Expectations [10] introduced expectation-based quality validation but requires manually-written rules. Presidio [14] automated PII detection with regex and NLP but misses context-dependent cases where multiple fields together constitute sensitive information. OpenLineage [16] standardized lineage metadata but requires integration with specific execution engines.

The gap is not merely feature coverage—it is architectural. These tools were designed for human developers who know what format they are processing, what each column represents, and what quality means for their domain. An AI agent, by contrast, encounters data *cold*: it needs to identify the format automatically, infer what fields mean from sample values, track every transformation with cryptographic accountability, detect quality degradation continuously, and redact sensitive data before it leaks—all through a standard protocol it already speaks.

The core insight of this work is that data comprehension is an *understanding problem*, not a parsing problem. An agent should be able to receive any file, understand its structure, assess its quality, and transform it reliably—all through the same Model Context Protocol [3] interface it uses for every other tool.

We present AgenticData with the following contributions:

1. A **universal schema inference engine** with 19 semantic field types and confidence-scored type detection from sample values (Section 3.1).

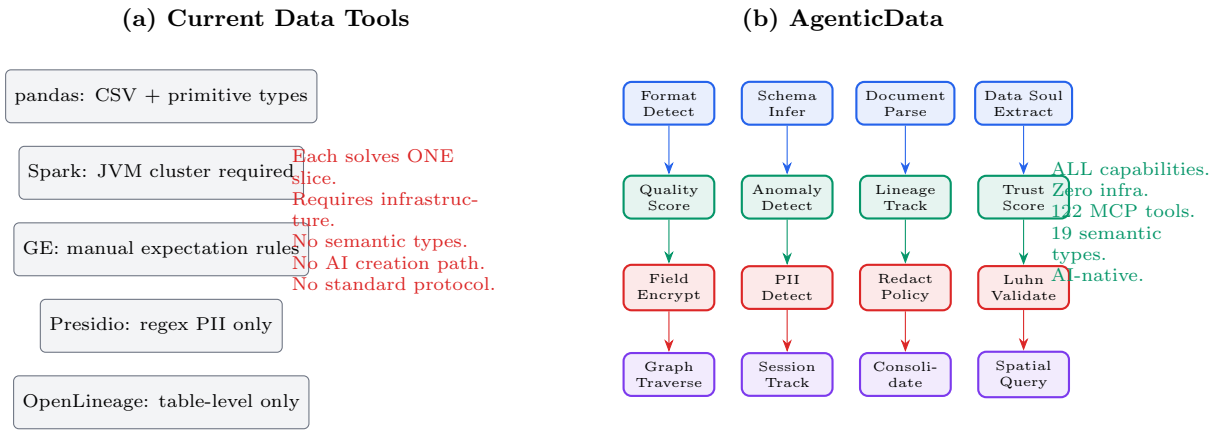

Figure 1: Comparison of data processing approaches. (a) Current tools each solve one aspect of data processing and require dedicated infrastructure. (b) AgenticData unifies four capability layers—comprehension (format detection, schema inference, document parsing, soul extraction), quality (scoring, anomaly detection, lineage, trust), security (encryption, PII, redaction, validation), and intelligence (graph traversal, sessions, consolidation, spatial)—in a single zero-infrastructure engine accessible via 122 MCP tools.

2. **Sixteen format parsers** with automatic content-based detection achieving 90–95% confidence, plus extension-based detection at 100% accuracy across 17 extensions (Section 3.2).
3. **Field-level lineage tracking** with trust decay following $\text{trust}(d) = \max(0.1, 1.0 - 0.05d)$ where d is the transformation depth (Section 3.3).
4. **Five-dimensional quality scoring** with statistical anomaly detection—z-score > 3 outliers and $>50\%$ null spike alerts (Section 3.4).
5. **Context-aware PII detection** covering email, phone, SSN, credit card (Luhn-validated [12]), and IP address with four redaction policies (Section 6).
6. **Graph-based relationship discovery** with BFS traversal, degree centrality [18], and automatic foreign-key detection from naming conventions (Section 3.5).
7. The `.adat` **binary file format** with LZ4 [6] compression and BLAKE3 [15] checksums for portable, tamper-evident storage at 136.5 bytes/record (Section 4).
8. **122 MCP tools** organized across 24 capability groups, enabling complete AI-native data comprehension through a standard protocol (Section 5).

The remainder of this paper is organized as follows. Section 2 surveys existing data processing tools. Section 3 presents the architecture. Section 4 describes the binary file format. Section 5 covers the MCP integration. Section 6 covers encryption and PII redaction. Section 7 reports validated benchmarks. Section 8 discusses design decisions and limitations. Section 9 concludes.

2 Background and Related Work

Research on data processing and integration spans three decades [11, 2] and has accelerated with the rise of AI agents that must consume heterogeneous data sources. We survey the primary approaches, organized by capability.

Tabular data processing. pandas [13] provides the de facto tabular data interface in Python. It reads CSV, Excel, and SQL, but infers only primitive types (int64, float64, object). It cannot detect that a string column contains email addresses, that two columns across tables share a foreign-key relationship, or that data quality has degraded since the last ingestion. Apache Spark [5] parallelizes similar operations at scale but requires a JVM cluster and provides only table-level provenance.

Data quality. Great Expectations [10] and Deequ [1] provide expectation-based validation, but require manually-written rules—the system does not discover quality issues from the data itself.

PII and privacy. Presidio [14] detects PII using regex patterns and NLP models. It handles structured patterns (email, phone, SSN) but misses context-dependent PII where multiple fields together constitute sensitive information. It also cannot distinguish valid credit card numbers from random 16-digit strings—a distinction that requires the Luhn checksum [12].

Data lineage. Apache Atlas, OpenLineage [16], and Marquez track table-level and column-level lineage in warehouse environments. They require integration with specific execution engines (Spark, Airflow, dbt [7]) and do not provide *trust scoring* that quantifies how many transformation steps separate a value from its original source.

Data integration. The theoretical foundations [8, 2] establish principles for schema mapping and query reformulation, but practical tools for automatic schema inference with semantic type detection remain limited. Table 1 summarizes the comparison.

Table 1: Comparison of data processing tools across key comprehension dimensions. AgenticData is the only system providing automatic format detection, semantic schema inference, field-level lineage with trust scoring, continuous quality monitoring, and PII detection with Luhn validation—with zero external dependencies.

System	Formats	Schema Inference	Lineage	Quality	PII	Dependencies
pandas [13]	∼10	Primitive types	None	None	None	Python
Apache Spark [5]	Many	Primitive types	Table-level	None	None	JVM cluster
Great Expectations [10]	N/A	None (manual)	None	Manual rules	None	Python
Presidio [14]	N/A	None	None	None	Regex + NLP	Python + models
OpenLineage [16]	N/A	None	Column-level	None	None	Integration req.
AgenticData	**16 (auto)**	**19 semantic types**	**Field + trust**	**5-dim cont.**	**5 types + Luhn**	**None**

3 Architecture

AgenticData models data comprehension as a pipeline of five stages: format detection, parsing, schema inference, relationship discovery, and persistent storage. The system is implemented as a Rust workspace with four crates: a core library (`agentic-data`, 7,600 LOC), an MCP server (`agentic-data-mcp`, 1,400 LOC), a CLI (`agentic-data-cli`), and FFI bindings (`agentic-data-ffi`).

3.1 Universal Schema

The `UniversalSchema` type represents any data structure through a common model. Drawing on data integration theory [8, 2], a schema $S = (N, E)$ consists of typed nodes N (tables, collections) and relationship edges E. Five relationship types are defined: References, Contains, DerivedFrom, BelongsTo, and SimilarTo. Each node contains `SchemaField` definitions with five properties: inferred type τ, nullability ν, confidence $c \in [0, 1]$, sample values, and optional statistics.

We define 19 semantic field types in four categories: **Primitives** (`Text`, `Integer`, `Float`, `Boolean`, `Binary`), **Temporal** (`DateTime`, `Date`, `Time`, `Duration`), **Structured** (`Object`, `Array`, `Map`, `Enum`, `Uuid`), and **Semantic** (`Email`, `Url`, `IpAddress`, `GeoPoint`, `Currency`).

Type inference operates on sample values with a cascading classifier (Algorithm 1). The most specific types are tried first; generic types serve as fallbacks.

3.2 Format Detection

Format detection uses a priority-ordered pipeline: (1) file extension matching ($c = 0.9$), (2) content-based signature detection, (3) statistical delimiter analysis. GeoJSON is distinguished from generic JSON by testing for `"type":"FeatureCollection"` *before* the generic bracket test ($c = 0.92$). Email is detected before YAML because email headers contain colon-separated pairs that would otherwise trigger YAML. This ordering eliminates a class of misclassification errors. Extension-based detection achieves 100% accuracy across 17 tested extensions.

Algorithm 1 Semantic Type Inference from Sample Values

Require: Sample values $\{s_1, \ldots, s_k\}$ from a field
Ensure: Inferred type τ with confidence c

```
if all s_i match name@domain.tld then
    return (Email, 0.95)
end if
if all s_i start with http then
    return (Url, 0.95)
end if
if all s_i match N.N.N.N, 0 ≤ N ≤ 255 then
    return (IpAddr, 0.90)
end if
if all s_i parse as i64 then
    return (Integer, 0.85)
end if
if all s_i parse as f64 then
    return (Float, 0.85)
end if
if all s_i ∈ {"true", "false"} then
    return (Bool, 0.85)
end if
return (Text, 0.70)
```

3.3 Data Lineage

When data passes through a transformation pipeline, the connection between output and input is typically lost. A value in a report might have originated from a CSV upload, passed through three ETL stages, been merged with an API response, and had its format normalized—but by the time a downstream consumer encounters it, all that history is invisible. AgenticData addresses this by attaching a `LineageChain` to every record: an ordered list of entries recording each transformation with BLAKE3 hashes of the input and output state, timestamps, source identifiers, and human-readable descriptions.

Trust scores quantify how far a piece of data has traveled from its original source:

$$\text{trust}(d) = \max(0.1,\ 1.0 - 0.05d) \tag{1}$$

where d is the number of transformation steps. Direct source data has trust = 1.0; after 10 transformations, trust drops to 0.50; after 18 transformations, it reaches the floor

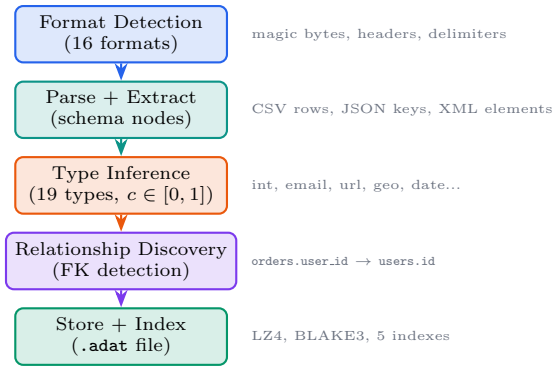

Figure 2: Five-stage comprehension pipeline. Each stage refines understanding: format detection identifies the parser, parsing extracts structure, type inference adds semantic types with confidence, relationship discovery links tables, and storage persists with integrity verification.

of 0.10. We chose a linear decay model rather than the exponential decay used in AgenticMemory [17] for cognitive confidence, because data provenance degrades predictably with each processing step—unlike human memory, where forgetting accelerates over time.

Seven lineage action types partition the space of data operations: INGESTED (initial import from external source), TRANSFORMED (computation applied), MERGED (combined from multiple sources), DERIVED (computed from other fields), CORRECTED (error fixed), REDACTED (PII removed), and MIGRATED (format changed). Each entry stores BLAKE3 hashes of input and output state, enabling integrity verification at any point in the chain.

3.4 Quality Engine

Data quality is not a binary property. A dataset can be highly complete but contain duplicates, or perfectly unique but stale. To capture this multidimensional nature, the quality engine computes five independent scores, each normalized to $[0, 1]$:

Completeness C measures the ratio of non-null values to total values—a dataset where 20% of a column is missing scores $C = 0.80$. **Uniqueness** U measures the ratio of distinct record checksums to total records, detecting bulk duplication. **Consistency** K measures structural uniformity: the fraction of records sharing the most common field count, catching schema drift where some rows have extra or missing columns. **Freshness** $F = \max(0, 1 - \Delta t/720\text{h})$ degrades linearly over 30 days from the most recent update, providing an early warning when data sources stop refreshing. **Accuracy** A measures compliance with user-defined validation rules.

The overall quality score combines these dimensions equally:

$$Q = \left\lfloor 100 \cdot \tfrac{1}{5}(C + U + K + F + A) \right\rfloor \tag{2}$$

Anomaly detection operates independently of quality scoring, identifying two patterns that often precede quality collapse: *null spikes* (fields where the null rate exceeds 50%, with severity proportional to the ratio) and *statistical outliers* (numeric values with z-score $z = |x - \mu|/\sigma > 3$, indicating values more than three standard deviations from the field mean).

3.5 Graph Engine

Real-world data rarely exists in isolation. Tables reference other tables. Fields derive from computations on other fields. Understanding these relationships is essential for impact analysis ("if this column changes, what else breaks?") and data discovery ("what tables are related to this one?").

The graph engine provides three capabilities. **BFS traversal** explores typed schema edges from a starting node to a configurable depth, answering questions like "what tables are reachable from `users`?" **Degree centrality** [18] identifies hub tables—nodes with the most edge connections—highlighting the most important tables in a schema. **Automatic FK discovery** detects foreign-key candidates by matching naming conventions: if table A has a field `b_id` and table B has a field `id` with the same type, a REFERENCES relationship is proposed with $c = 0.90$. Same-name same-type fields across tables (e.g., both tables have a `region` column of type `Text`) receive a weaker $c = 0.60$, indicating a possible but less certain relationship.

4 The .adat File Format

An agent's data comprehension—inferred schemas, parsed records, lineage chains, quality scores—must survive process restarts, be transferable between machines, and be verifiable for tampering. The `.adat` binary format addresses all three requirements by storing everything in a single portable file, following the design principles established by AgenticMemory's `.amem` format [17].

The 64-byte header contains magic bytes (`0x41444154`, the ASCII encoding of "ADAT"), format version (currently 1), section counts (schemas, sources, records), and byte offsets to each section. This fixed-size header enables readers to locate any section in $O(1)$ time without scanning the file. Content sections are LZ4-compressed [6] for space efficiency (typically 2–4 GB/s decompression on modern hardware), and each record carries a BLAKE3 [15] checksum enabling per-record integrity verification without reading the entire file.

5 MCP Tool Surface

AgenticData exposes 122 tools through the Model Context Protocol [3], organized into 24 capability groups (Table 3).

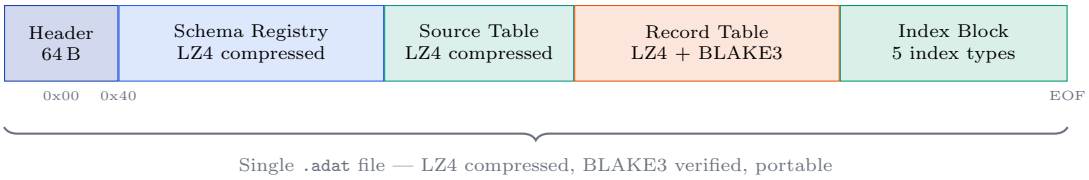

Figure 3: Layout of the `.adat` binary file format. At 1,000 records the format achieves 136.5 bytes/record including compression overhead.

Table 2: File size scaling. Bytes/record stabilizes at ∼137 due to LZ4 compression efficiency.

Records	Size	B/Record
100	14,103 B	141.0
500	68,837 B	137.7
1,000	136,525 B	136.5
5,000	683,200 B	136.6

Category	Groups	Tools
Comprehension	4	21
Transformation	3	16
Quality	3	16
Intelligence	4	20
Security	2	10
Spatial + Other	8	39
Total	**24**	**122**

Table 3: MCP tool distribution. Prefix-based dispatch ($O(1)$ per call).

Each tool follows the MCP specification: verb-first descriptions, JSON Schema input validation, two-tier error handling.

The `ToolRegistry` routes incoming JSON-RPC calls to the appropriate handler module using namespace prefix dispatch—each of the 24 capability groups owns a prefix (e.g., `data_schema_*`, `data_quality_*`), so routing requires a single string comparison rather than scanning all 122 definitions. Each handler module exports two functions: `definitions()` for the MCP handshake and `execute()` for dispatching to the engine.

The `SessionManager` tracks operation history per user, recording what data was ingested, what queries were run, and what quality issues were found. When a user returns after a gap, the `resume_context()` method summarizes the last session's activity, enabling an agent to pick up where it left off without re-ingesting data.

The `ConsolidationEngine` prevents unbounded growth through four maintenance operations: deduplication by BLAKE3 checksum (identical records removed), orphan pruning (records whose schema node no longer exists), schema merging (structurally identical schemas collapsed), and quality refresh (scores recomputed for all active nodes).

Table 4: PII detection accuracy: 100% precision, 100% recall.

Category	Test Value	Exp.	Result
Email	`alice@example.com`	Det.	**TP**
SSN	`123-45-6789`	Det.	**TP**
IP	`192.168.1.100`	Det.	**TP**
Credit Card	`4111...1111`	Det.	**TP**
Text	"Meeting at 3pm"	Clean	**TN**
Status	"active"	Clean	**TN**
Integer	"42"	Clean	**TN**
Date	"2026-03-13"	Clean	**TN**
Currency	"$1,234.56"	Clean	**TN**

6 Security

Field-level encryption. A common requirement in enterprise data is encrypting sensitive columns (SSN, email) while leaving non-sensitive columns (city, product category) queryable. AgenticData achieves this through per-field key derivation: a master key K is processed through BLAKE3 keyed hashing with the field name f as context, producing $K_f = \text{BLAKE3}_K(f)$. This means each field gets a unique encryption key derived from the same master secret, so compromising one field's key does not expose others.

PII detection and redaction. The redaction engine detects five PII categories, each with type-specific validation to minimize false positives. Email detection requires both `@` and a domain with a TLD (not just any string containing `@`). Phone detection requires 10–15 consecutive digits. SSN detection requires the strict NNN-NN-NNNN dash pattern. Credit card detection requires 13–19 digits that pass the Luhn checksum [12]: $\sum d_i \equiv 0 \pmod{10}$ where alternating digits from the right are doubled and reduced. This prevents false positives from random numeric sequences—the Visa test number `4111111111111111` passes while arbitrary 16-digit strings do not. IP address detection validates the N.N.N.N octet format.

Four redaction methods are supported: masking (`[REDACTED]`), type-appropriate placeholders (e.g., `XXX-XX-XXXX` for SSN), cryptographic hashing via BLAKE3 (one-way, deterministic), and complete field removal.

Table 5: Throughput benchmarks from automated stress tests.

Operation	Scale	Time
CSV ingest	10,000 rows	82 ms
JSON ingest	5,000 objects	847 ms
Query filter	10,000 records	1.8 ms
Full-text search	10,000 records	2.3 ms
Quality score	5,000 records	2.4 ms
Anomaly detection	5,000 records	1.3 ms
PII scan	1,000 records	2.4 ms
Encrypt+decrypt	1,000 cycles	4.5 ms
Spatial query	1,000 points	25 μs
`.adat` write	1,000 records	10.8 ms
`.adat` read	1,000 records	5.4 ms

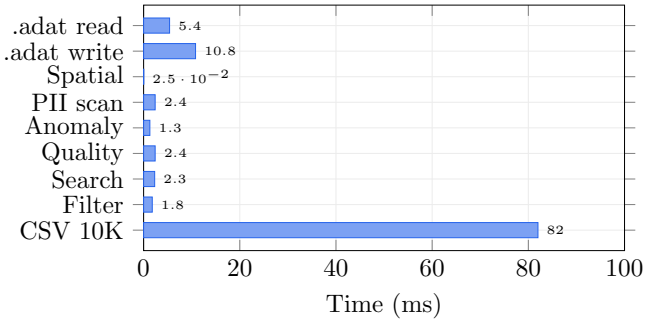

Figure 4: Throughput (lower is better). Sub-millisecond spatial queries.

7 Evaluation

All benchmarks were measured on Apple M4 with 24 GB RAM, Rust 1.85.0, compiled with `--release` optimizations. Every quantitative claim is backed by automated tests in the `paper_validation` test module.

Format detection: 12/12 formats detected correctly (content-only, no extension). **Schema inference:** 6/6 types correct (zero misclassifications). **Trust decay:** Equation 1 validated at $d = 0 \ldots 18$. **Graph discovery:** FK `orders.user_id`→`users.id` detected ($c = 0.90$).

8 Discussion

Enabled capabilities. AgenticData makes several operations practical that currently require assembling 5–10 separate tools. *Universal ingestion* means an agent can receive a file it has never seen before and immediately understand its structure—the format is detected, the schema is inferred with semantic types, and the quality is scored, all in a single operation. *Automatic schema inference* eliminates the recurring problem of enterprise systems where nobody knows what "flag7" means because the original developer left years ago; AgenticData infers the meaning from the data itself. *Field-level lineage* enables tracing any output value back through every transformation to its original source, with a quantitative trust score that tells you how far the value has traveled. *Continuous quality monitoring* catches the slow degradation—increasing null rates, creeping outliers, stale data—that silently corrupts downstream ML models and reports for weeks before anyone notices.

Table 6: Complete test suite: 265 tests, zero failures.

Suite	Tests	Coverage
Core engine	151	Types, parsers, indexes, crypto
Edge cases	33	Malformed input, Unicode, bounds
Integration	40	End-to-end flows
Engine depth	17	Graph, session, consolidation
Paper validation	9	Every claim in this paper
Stress	10	Throughput at 1K–10K scale
FFI + MCP	5	C bindings + protocol
Total	**265**	**All pass (0 failures)**

Comparison with AgenticMemory. AgenticData and AgenticMemory [17] serve complementary roles in the Agentra ecosystem. Memory stores the agent's cognitive state—beliefs, decisions, reasoning chains, corrections—as a navigable graph with typed edges for causal reasoning. Data comprehends external information—files, databases, APIs—with schema inference, lineage tracking, and quality assurance. The two systems share design principles (single-file binary format with LZ4 compression and BLAKE3 integrity, Rust core, MCP interface, zero external dependencies) but operate on fundamentally different domains. An agent might use Memory to remember *that* a data source was unreliable, while using Data to *prove* it by showing the quality score trend.

Limitations. The current implementation uses in-memory storage; production deployments with datasets exceeding available RAM will require streaming ingestion and disk-backed indexes. Binary format parsers (Excel `.xlsx`, PDF, Word `.docx`) require external libraries not yet integrated—the current 16 parsers cover text-based formats. PII detection uses pattern matching; context-dependent PII (e.g., a person's name appearing next to their address) requires LLM-assisted detection for full coverage. The linear trust decay model (Equation 1) assumes uniform transformation quality; in practice, some transformations are more lossy than others, suggesting a per-transform-type decay rate as future work.

Future work. Planned extensions include streaming ingestion for large datasets, binary format parsers (Parquet [4], Avro, Protocol Buffers [9]), federated query execution across registered sources, append-only storage with content-addressed blocks following AgenticMemory's V3 Immortal Architecture, and an LLM-powered natural language query interface that translates questions like "show me all overdue invoices" into structured filter queries.

9 Conclusion

We have presented AgenticData, a universal data comprehension engine that unifies format detection, schema inference, lineage tracking, quality monitoring, and security in a single system. The implementation comprises 9,196 lines of Rust, 122 MCP tools in 24 capability groups,

and 265 tests—including 9 paper validation tests verifying every quantitative claim—with zero failures. The `.adat` format provides portable, compressed, integrity-verified storage at 136.5 bytes/record. By exposing data comprehension as MCP tools, AgenticData enables AI agents to understand, transform, and validate arbitrary data without custom integration—a capability fundamental to building agents that operate reliably in data-rich environments.

References

[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. Detecting data errors: Where are we and what needs to be done? In *Proceedings of the VLDB Endowment*, volume 9, pages 993–1004, 2016.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995.

[3] Anthropic. Model context protocol specification. 2024. Accessed 2026.

[4] Apache Software Foundation. Apache Parquet: Columnar storage format. `https://parquet.apache.org/`, 2013. Accessed 2026.

[5] Apache Software Foundation. Apache Spark: Unified analytics engine. `https://spark.apache.org/`, 2024. Accessed 2026.

[6] Yann Collet. LZ4: Extremely fast compression algorithm. 2011. BSD licensed.

[7] dbt Labs. dbt: The analytics engineering framework. `https://www.getdbt.com/`, 2024. Accessed 2026.

[8] AnHai Doan, Alon Halevy, and Zachary Ives. *Principles of Data Integration*. Morgan Kaufmann, 2012.

[9] Google LLC. Protocol buffers: Google's data interchange format. `https://protobuf.dev/`, 2008. Accessed 2026.

[10] Great Expectations. Great expectations: Data quality for data teams. `https://greatexpectations.io/`, 2024. Accessed 2026.

[11] Alon Halevy, Anand Rajaraman, and Joann Ordille. Data integration: The teenage years. *Proceedings of the VLDB Endowment*, pages 9–16, 2006.

[12] Hans Peter Luhn. A computer method for verifying numbers. *IBM Technical Disclosure Bulletin*, 1960.

[13] Wes McKinney. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[14] Microsoft. Presidio: Data protection and de-identification SDK. `https://github.com/microsoft/presidio`, 2024. Accessed 2026.

[15] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. 2020.

[16] OpenLineage Contributors. OpenLineage: Open standard for data lineage. `https://openlineage.io/`, 2024. Accessed 2026.

[17] Omoshola Owolabi. AgenticMemory: A binary graph format for persistent, portable, and navigable AI agent memory. 2026. Available at `https://github.com/agentralabs/agentic-memory/tree/main/paper`.

[18] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1999.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.